

Basic processing of biological sequences

BioInf 2025/2026 1st Semester

André Lamúrias

a.lamurias@fct.unl.pt

Summary

- Representations and Basic Algorithms
- Transcription and Reverse Complement
- Translation
- Input and Output
- A Class for Biological Sequences
- Biopython

Recap

- Biological data is sequential (DNA, RNA, proteins)
- Biological processes transform DNA->RNA->proteins (Central Dogma of Biology)

DNA and RNA molecules

- Represented as one-dimensional sequences of nucleotides
- Alphabet of four distinct symbols
 - DNA: A (adenine), C (cytosine), G (guanine), and T (thymine)
 - RNA: A (adenine), C (cytosine), G (guanine), and U (uracil)
- Extended set of symbols defined by International Union of Pure and Applied Chemistry (IUPAC)
 - allow ambiguity in the identification of a nucleotide
 - uncertainty in the identification of the nucleotide

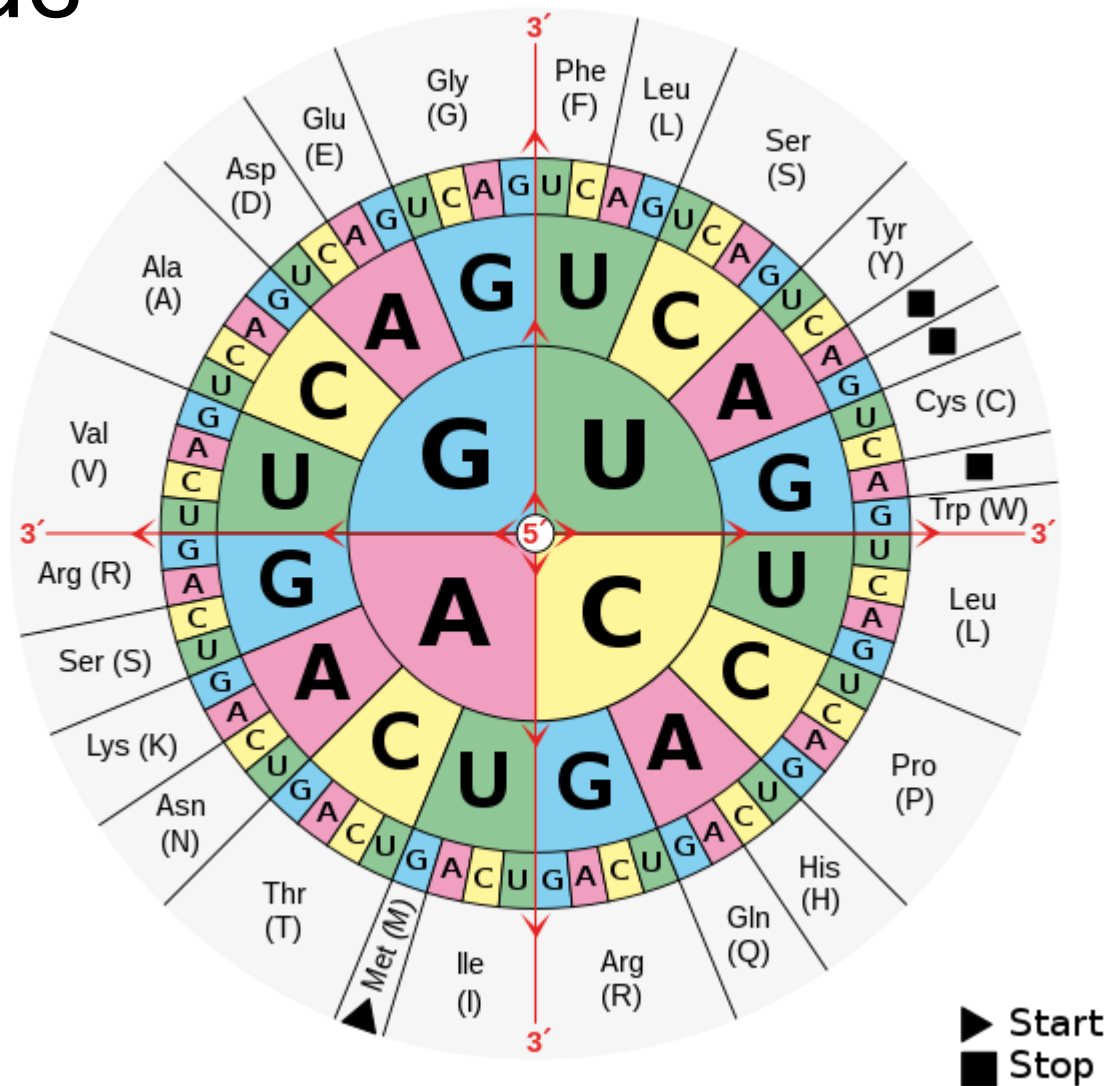
DNA and RNA molecules

Symbol	Name	Nucleotides represented
A	Adenine	A
C	Cytosine	C
G	Guanine	G
T	Thymine	T
U	Uracil	U
K	Keto	G, T
M	Amino	A, C
R	Purine	A, G
S	Strong	C, G
W	Weak	A, T
Y	Pyrimidine	C, T
B	Not A	C, G, T
D	Not C	A, G, T
H	Not G	A, C, T
V	Not T	A, C, G
N	Any base	A, C, G, T

Proteins

- Represented as one-dimensional sequences of aminoacids
- 20 different aminoacids can occur in protein sequences
- Symbol for stop codon
 - used for instance when running automatic tools for DNA translation
 - the most used symbols for the stop codon are the underscore (_) or the asterisk (*)

Genetic code



Proteins

Symbol	Name
A	Alanine
C	Cysteine
D	Aspartic acid
E	Glutamic acid
F	Phenylalanine
G	Glycine
H	Histidine
I	Isoleucine
L	Lysine
M	Methionine
N	Asparagine
P	Proline
Q	Glutamine
R	Arginine
S	Serine
T	Threonine
V	Valine
T	Tryptophan
Y	Tyrosine

Basic Algorithms

- Calculate the frequency of the different symbols in the sequences

```
def frequency(seq):  
    """ Calculates the frequency of each symbol in the sequence.  
        Returns a dictionary. """  
    dic = {}  
    for s in seq.upper():  
        if s in dic: dic[s] += 1  
        else: dic[s] = 1  
    return dic
```

```
In : frequency("atagataactcgcatag")  
Out: {'A': 7, 'T': 4, 'G': 3, 'C': 3}
```

```
In : frequency("MVVMKKSHHVLHSQSLIK")  
Out: {'M': 2, 'V': 3, 'K': 3, 'S': 3, 'H': 3, 'L': 2, 'Q': 1, 'I': 1}
```

Basic Algorithms

- Compute the GC content of a DNA sequence, i.e. the percentage of 'G' and 'C' nucleotides in the sequence

```
def gc_content(dna_seq):  
    """ Returns percentage of G and C nucleotides in a DNA sequence."""  
    gc_count = 0  
    for s in dna_seq:  
        if s in "GCgc": gc_count += 1  
    return gc_count / len(dna_seq)
```

```
In : gc_content("atagataactcgcatag")  
Out: 0.35294117647058826
```

```
In : gc_content("ATGAAGGCCATTGTAATGGGCCGC")  
Out: 0.5416666666666666
```

Basic Algorithms

- Compute the GC content of the non-overlapping sub-sequences of size k of the inputted sequence

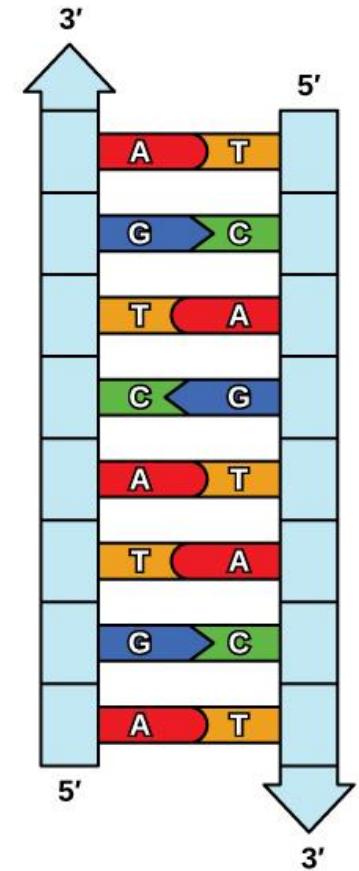
```
def gc_content_subseq(dna_seq , k=100):  
    """ Returns GC content of non-overlapping sub-sequences of size k.  
        The result is a list. """  
    res = []  
    for i in range(0, len(dna_seq)-k+1, k):  
        subseq = dna_seq[i:i+k]  
        gc = gc_content(subseq)  
        res.append(gc)  
    return res
```

```
In : gc_content_subseq("atagataactcgcatag", 5)  
Out: [0.2, 0.2, 0.6]
```

```
In : gc_content_subseq("ATGAAGGCCATTGTAATGGGCCGC", 5)  
Out: [0.2, 0.8, 0.2, 0.6]
```

Transcription of DNA into RNA

- Fundamental step within protein synthesis
- DNA molecules have **two complementary strands** that are **read in reverse directions**.
- Only one of the strands is used to create RNA molecule
 - The two strands are split
 - The new RNA molecule is complementary to the coding strand
 - It is similar to the other strand, but T→U



<https://opentextbc.ca/biology/chapter/9-2-dna-replication/>

Transcription of DNA into RNA

- Compute the RNA corresponding to the transcription of coding DNA strand

```
def transcription(dna_seq):  
    """ Function that computes the RNA corresponding to the  
        transcription of the DNA sequence provided. """  
    return dna_seq.upper().replace("T","U")
```

```
In : transcription("atagataactcgcatag")  
Out: 'AUAGAUAACUCGCAUAG'
```

```
In : transcription("ATGAAGGCCATTGTAATGGGCCGC")  
Out: 'AUGAAGGCCAUUGUAAUGGGCCGC'
```

Transcription of DNA into RNA

- If we have **ACTGAG** in one of the strands, going 5' to 3', what is the other strand?
- Now reverse it so that it is also in the 5' to 3' direction

Reverse Complement

- In Bioinformatics databases usually there is only the sequence of one of the strands
- We can easily compute the Reverse Complement

Reverse Complement

- Compute the reverse complement of a DNA sequence.

```
def reverse_complement(dna_seq):  
    """ Computes the reverse complement of the DNA sequence. """  
    comp = ""  
    for c in dna_seq.upper():  
        if c == "A":  
            comp = "T" + comp  
        elif c == "T":  
            comp = "A" + comp  
        elif c == "G":  
            comp = "C" + comp  
        elif c == "C":  
            comp = "G" + comp  
    return comp
```

```
In : reverse_complement("ATGAAGGCCATTGTAATGGGCCGC")  
Out: 'GCGGCCCATTCACAAATGGCCTTCAT'
```


Translation

- Proteins = chains of aminoacids (AA)
- AAs determined by information contained within messenger RNA molecules
 1. Split DNA sequence into non-overlapping sub-sequences of size=3 (codons);
 2. for each codon, use conversion table to get corresponding AA;
 3. AAs are gathered, keeping their order, in the resulting protein sequence.

Translation

- Mapping between codons and aminoacids

Nucleotides	Amino acids
UUU, UUC	Phenylalanine/ Phe / F
UUA, UUG, UCU, UCA, UCC, UCG	Leucine / Leu / L
AUU, AUC, AUA	Leucine / Leu / L
AUG	Methionine / Met / M (start)
GUU, GUC, GUA, GUG	Valine / Val / V
UCU, UCC, UCA, UCG, AGA, AGG	Serine / Ser / S
CCU, CCC, CCA, CCG	Proline / Pro / P
ACU, ACC, ACA, ACG	Threonine / Thr / T
GCU, GCC, GCA, GCG	Alanine / Ala / A
UAA, UAC	Tyrosine / Tyr / Y
UAA, UAG, UGA	Stop codons
CAU, CAG	Histidine / His / H
CAA, CAG	Glutamine / Gln / Q
AAU, AAC	Asparagine / Asn / N
AAA, AAG	Lysine / Lys / K
GAU, GAC	Aspartic Acid / Asp / D
GAA, GAG	Glutamic Acid / Glu / G
UGU, UGC	Cysteine / Cys / C
UGG	Arginine / Arg / R
CGU, CGC, CGA, CGG, AGA, AGG	Glycine / Gly / G

Translation

- Translate a codon to the corresponding aminoacid. _ denotes a stop codon.

```
def translate_codon(cod):
    """Translates a codon into an aminoacid using an internal
    dictionary with the standard genetic code."""
    tc = {"GCT":"A", "GCC":"A", "GCA":"A", "GCG":"A",
          "TGT":"C", "TGC":"C",
          "GAT":"D", "GAC":"D",
          "GAA":"E", "GAG":"E",
          "TTT":"F", "TTC":"F",
          "GGT":"G", "GGC":"G", "GGA":"G", "GGG":"G",
          "CAT":"H", "CAC":"H",
          "ATA":"I", "ATT":"I", "ATC":"I",
          "AAA":"K", "AAG":"K",
          "TTA":"L", "TTG":"L", "CTT":"L", "CTC":"L", "CTA":"L", "CTG":"L",
          "ATG":"M", "AAT":"N", "AAC":"N",
          "CCT":"P", "CCC":"P", "CCA":"P", "CCG":"P",
          "CAA":"Q", "CAG":"Q",
          "CGT":"R", "CGC":"R", "CGA":"R", "CGG":"R", "AGA":"R", "AGG":"R",
          "TCT":"S", "TCC":"S", "TCA":"S", "TCG":"S", "AGT":"S", "AGC":"S",
          "ACT":"T", "ACC":"T", "ACA":"T", "ACG":"T",
          "GTT":"V", "GTC":"V", "GTA":"V", "GTG":"V",
          "TGG":"W",
          "TAT":"Y", "TAC":"Y",
          "TAA":"_", "TAG":"_", "TGA":"_"}
    if cod in tc: return tc[cod]
    else: return None
```

Translation

```
def translate_seq(dna_seq , ini_pos = 0):  
    """ Translates a DNA sequence into an aminoacid sequence. """  
    seqm = dna_seq.upper()  
    seq_aa = ""  
    for pos in range(ini_pos , len(seqm)-2,3):  
        cod = seqm[pos:pos+3]  
        seq_aa += translate_codon(cod)  
    return seq_aa
```

```
In : translate_seq("ATGAAGGCCATTGTAATGGGCCGC")  
Out: 'MKAIVMGR'
```

```
In : translate_seq("ATGAAGGCCATTGTAATGGGCCGC",2)  
Out: 'EGHCNGP'
```

```
In : translate_seq("tgattaactcgcatacg",1)  
Out: 'D_LAY'
```

Translation

- Genetic code is redundant: a single aminoacid may be encoded by different codons.
- Codon usage provides interesting statistics when applied to the genes of different species.

Translation

- Compute the codon usage, providing an aminoacid and a DNA coding sequence.

```
def codon_usage(dna_seq , aa):  
    """Provides the frequency of each codon encoding a given  
        aminoacid, in a DNA sequence."""  
    seqm = dna_seq.upper()  
    dic = {}  
    total = 0  
    for i in range(0, len (seqm)-2, 3):  
        cod = seqm[i:i+3]  
        if translate_codon(cod) == aa:  
            if cod in dic: dic[cod] += 1  
            else: dic[cod] = 1  
            total += 1  
    if total >0:  
        for k in dic:  
            dic[k] /= total  
    return dic
```

Translation

- Compute the codon usage, providing an aminoacid and a DNA coding sequence.

```
In : translate_seq("TTACGCCTTATTGTATTAGGCCGC")
```

```
Out: 'LRLIVLGR'
```

```
In : codon_usage("TTACGCCTTATTGTATTAGGCCGC", "L")
```

```
Out: {'TTA': 0.6666666666666666, 'CTT': 0.3333333333333333}
```

```
In : codon_usage("TTACGCCTTATTGTATTAGGCCGC", "R")
```

```
Out: {'CGC': 1.0}
```

```
In : codon_usage("TTACGCCTTATTGTATTAGGCCGC", "A")
```

```
Out: {}
```

Input and Output

- Read a sequence from a text file, where the sequence may be given in multiple lines.

```
def read_seq_from_file(filename):  
    """ Reads a sequence from a multi-line text file. """  
    fh = open(filename , "r")  
    lines = fh.readlines()  
    seq = ""  
    for l in lines:  
        seq += l.replace("\n","")  
    fh.close()  
    return seq
```

- Write a sequence to a text file.

```
def write_seq_to_file(seq, filename):  
    """ Writes a sequence to file. """  
    fh = open(filename , "w")  
    fh.write(seq)  
    fh.close()
```


Input and Output

- Using these functions, we can redefine the previous function to read the input sequence from a given file and write the results in different files.

```
def sequence_from_file():  
    """Reads a DNA sequence from file. Verify validity. Compute  
        gc content. Write transcription, reverse complement and  
        translation to new files. """  
    fname = input("Insert input filename:")  
    seq = read_seq_from_file(fname)  
    if validate_dna(seq):  
        print("Valid sequence")  
        print("GC content (global):", gc_content(seq))  
        write_seq_to_file(transcription(seq), "trs-" + fname)  
        write_seq_to_file(reverse_complement(seq), "rc-" + fname)  
        write_seq_to_file(translate_seq(seq), "trl-" + fname)  
    else: print("DNA sequence is not valid")
```

Input and Output

- FASTA format:

```
>NG_008679.1:5001-38170 Homo sapiens paired box 6 (PAX6)
ACCTCTTTTCTTATCATTGACATTTAACTCTGGGGCAGGTCTCGCGTAGAACGCGGCTGTCAGATCT
GCCACTTCCCCTGCCGAGCGGCGGTGAGAAGTGTGGGAACCGGCGCTGCCAGGCTCACCTGCCTCCCCGC
CCTCCGCTCCCAGGTAACCGCCCGGGCTCCGGCCCCGGCCCGGCTCGGGGCGCGGGGCTCTCCGCTG
CCAGCGACTGCTGTCCCCAAATCAAAGCCCCGCCCAAGTGGCCCCGGGGCTTGATTTTGTCTTTAAAAG
GAGGCATACAAAGATGGAAGCGAGTTACTGAGGGAGGGATAGGAAGGGGGTGGAGGAGGACTTGTCTT
TGCCGAGTGTGCTCTTCTGCAAAAGTAGCAAAATGTTCCACTCCTAAGAGTGGACTTCCAGTCCGGCCCT
GAGCTGGGAGTAGGGGGCGGGAGTCTGCTGCTGCTGTCTGCTAAAGCCACTCGCGACCGCGAAAAATGCA
GGAGGTGGGGACGCACTTTGCATCCAGACCTCCTCTGCATCGCAGTTCACGACATCCACGCTTGGGAAAG
TCCGTACCCGCGCCTGGAGCGCTTAAAGACACCCTGCCGCGGGTTCGGGCGAGGTGCAGCAGAAGTTCCC
GCGGTTGCAAAGTGCAGATGGCTGGACCGCAACAAAGTCTAGAGATGGGGTTCGTTTCTCAGAAAGACGC
```

- FASTQ:

```
Identifier | @HWI-EAS209_0006_FC706VJ:5:58:5894:21141#ATCACG/1
Sequence  | TTAATTGGTAAATAAATCTCCTAATAGCTTAGATNTTACCTNNNNNNNNNTAGTTTCTTGAGA
+ sign & identifier | +HWI-EAS209_0006_FC706VJ:5:58:5894:21141#ATCACG/1
Quality scores | efcffffcfeeffcfffffdff`feed]`_]_Ba^__[YBBBBBBBBBRT\]] [] dddd`

Base T
phred Quality ] = 29
```

<https://compgenomr.github.io/book/fasta-and-fastq-formats.html>

A Class for Biological Sequences

- In python:

```
class ClassName:  
    """ Optional documentation """  
    --body_of_the_class--
```

- As a convention:
 - Class name: first letter of each word is represented in uppercase.
 - Attribute names: lowercase with underscores as for variable names.
- Constructor method: `__init__`
 - Automatically invoked when new objects of this class are declared
 - Defines the initial state of the objects that are instances of this class.

A Class for Biological Sequences

- A simple class to represent and process biological sequences.

```
class MySeq:
    """ Class for Biological sequences. """
    def __init__(self, seq, seq_type = "DNA"):
        self.seq = seq
        self.seq_type = seq_type

    def print_sequence(self):
        print("Sequence: " + self.seq)

    def get_seq_biotype(self):
        return self.seq_type

    def show_info_seq(self):
        print("Sequence: " + self.seq + " biotype: " + self.seq_type)

    def count_occurrences(self, seq_search):
        return self.seq.count(seq_search)
```

A Class for Biological Sequences

- The attributes of a class can be directly modified.
- It may be better to validate such modification before it occurs and, therefore, be made through a method.

```
# class method to validate biotype update
def set_seq_biotype(self, bt):
    biotype = bt.upper()
    if biotype == "DNA" or biotype == "RNA" or biotype == "PROTEIN":
        self.seq_type = biotype
    else:
        print("Non biological sequence type!")
```

```
In : s1 = MySeq("ATAATGATAGATAGATGAT")
In : s1.set_seq_biotype("time series")
Non biological sequence type!
In : s1.set_seq_biotype("dna")
In : s1.show_info_seq()
Sequence: ATAATGATAGATAGATGAT biotype: DNA
```

A Class for Biological Sequences

- Several methods are shared across different classes.
 - E.g. the method `len` is common to a number of classes such as strings, lists or dictionaries.
- These methods that implement this type of generic functionality can be redefined for new classes.
- This implies re-implementing a set of special methods that have a very specific behavior and follow a pre-defined naming, all of them starting and ending with **double underscores** `__`.

A Class for Biological Sequences

```
def __len__(self):  
    return len(self.seq)  
  
def __str__(self):  
    return self.seq  
  
def __getitem__(self, n):  
    return self.seq[n]  
  
def __getslice__(self, i, j):  
    return self.seq[i:j]
```

```
In : s1 = MySeq("MKKVSJEMSSVPYW", "PROTEIN")  
In : print(s1)  
MKKVSJEMSSVPYW  
In : print(len(s1))  
14  
In : print(s1[4])  
S  
In : print(s1[2:5])  
KVS
```

A Class for Biological Sequences

- Method to return the allowed alphabet, given the sequence type.

```
def alphabet(self):  
    if (self.seq_type=="DNA"): return "ACGT"  
    elif (self.seq_type=="RNA"): return "ACGU"  
    elif (self.seq_type=="PROTEIN"): return "ACDEFGHIKLMNPQRSTVWY"  
    else: return None
```

```
In : s1 = MySeq("MKKVSJEMSSVPYW", "PROTEIN")  
In : s1.alphabet()  
Out: 'ACDEFGHIKLMNPQRSTVWY'  
In : s2 = MySeq("ATGAAGGCCATTGTAATGGGCCGC", "DNA")  
In : s2.alphabet()  
Out: 'ACGT'
```


A Class for Biological Sequences

- Method to validate sequences, taking into account the sequence type.

```
def validate(self):  
    alp = self.alphabet()  
    res = True  
    i = 0  
    while i < len(self.seq) and res:  
        if self.seq[i] not in alp: res = False  
        else: i += 1  
    return res
```

```
In : s1 = MySeq("MKKVSJEMSSVPYW", "PROTEIN")  
In : s1.validate()  
Out: False  
In : s2 = MySeq("ATGAAGGCCATTGTAATGGGCCGC", "DNA")  
In : s2.validate()  
Out: True  
In : s3 = MySeq("MKKVSEMSSVPYW", "PROTEIN")  
In : s3.validate()  
Out: True
```

A Class for Biological Sequences

- Method to compute the transcription of the DNA sequence

```
def transcription(self):  
    if (self.seq_type == "DNA"):  
        return MySeq(self.seq.replace("T","U"), "RNA")  
    else:  
        return None
```

```
In : s2 = MySeq("ATGAAGGCCATTGTAATGGGCCGC", "DNA")  
In : s2.show_info_seq()  
Sequence: ATGAAGGCCATTGTAATGGGCCGC biotype: DNA  
In : s3 = s2.transcription()  
In : s3.show_info_seq()  
Sequence: AUGAAGGCCAUUGUAAUGGGCCGC biotype: RNA
```

- The methods all return objects of the MySeq class, but with different types, according to the function's biological meaning.

Biopython

- Addresses a large number of bioinformatics tasks
- Open source, easily installed
- Object-oriented

Biopython

- Create a simple sequence object
- Biopython sequences are immutable
- Define sequence with generic alphabet:

```
>>> from Bio.Seq import Seq
>>> my_seq = Seq("ATAGAGAAATCGCTGC")
>>> my_seq
Seq('ATAGAGAAATCGCTGC', Alphabet())
>>> print (my_seq)
ATAGAGAAATCGCTGC
>>> my_seq.alphabet
Alphabet()
```

Biopython

- Specify the alphabet:

```
>>> from Bio.Alphabet import IUPAC
>>> my_seq = Seq("ATAGAGAAATCGCTGC", IUPAC.unambiguous_dna)
>>> my_seq
Seq('ATAGAGAAATCGCTGC', IUPACUnambiguousDNA())
>>> my_seq.alphabet
IUPACUnambiguousDNA()
>>> my_prot = Seq("MJKLKVERSVMSVLP", IUPAC.protein)
>>> my_prot
Seq('MJKLKVERSVMSVLP', IUPACProtein())
```

```
>>> IUPAC.unambiguous_dna.letters
'GATC'
>>> IUPAC.ambiguous_dna.letters
'GATCRYWSMKHBVDN'
>>> IUPAC.IUPACProtein.letters
'ACDEFGHIKLMNPQRSTVWY'
>>> IUPAC.ExtendedIUPACProtein.letters
'ACDEFGHIKLMNPQRSTVWYBXZJUO'
```

Biopython

- Seq objects can be treated as strings
- Only sequences with compatible alphabets can be concatenated
- Transcription and reverse complement are also implemented:

```
>>> coding_dna = Seq("ATGAAGGCCATTGTAATGGGCCGC", IUPAC.  
unambiguous_dna)  
>>> template_dna = coding_dna.reverse_complement()  
>>> template_dna  
Seq('GCGGCCCATTCACAATGGCCTTCAT', IUPACUnambiguousDNA())  
>>> messenger_rna = coding_dna.transcribe()  
>>> messenger_rna  
Seq('AUGAAGGCCAUUGUAAUGGGCCGC', IUPACUnambiguousRNA())
```

Biopython

- Translation is also implemented:

```
>>> rna_seq = Seq('AUGCGUUUAACU', IUPAC.unambiguous_rna)
>>> rna_seq.translate()
Seq('MRLT', IUPACProtein())
>>> coding_dna = Seq("ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG", IUPAC
.unambiguous_dna)
>>> coding_dna.translate()
Seq('MAIVMGR*KGAR*', HasStopCodon(IUPACProtein(), '*'))
>>> coding_dna.translate(table="Vertebrate Mitochondrial")
Seq('MAIVMGRWKGAR*', HasStopCodon(IUPACProtein(), '*'))
```

- We can use different codon table besides the “Standard” one
- Check Bio.Data.CodonTable for more information

Biopython

- Reading files – SeqIO class to store sequence and metadata
- Fasta format (.fa, .fasta, .fna): header (>) followed by sequence lines

```
>>> from Bio import SeqIO
>>> record = SeqIO.read("NC_005816.fna", "fasta")
>>> record
SeqRecord(seq=Seq("TGTAACGAACGGTGCAAT...")
>>> len(record.seq)
9609
>>> record.id
gi|45478711|ref|NC_005816.1|
>>> record.description
gi|45478711|ref|NC_005816.1| Yersinia pestis biovar Microtus str .
91001 plasmid pPCP1, complete sequence
```


Bibliography

- Rocha Chapter 4